

Conversational Geospatial Analytics: Bridging Google Earth Engine and Large Language Models via the Model Context Protocol

Madhulika Smriti Independent project · 2026

Abstract

Google Earth Engine (GEE) democratized planetary-scale geospatial analysis by offering a managed compute backend and a vast public data catalog, but its primary interfaces — the JavaScript Code Editor and the Python client library — still require users to translate every analytic question into code. As the same patterns are written repeatedly across sessions (load collection, filter by date and AOI, compute index, reduce, export), the cognitive overhead of script authorship becomes a recurring tax on practitioner productivity. This paper presents **GEE_MCP_Plugin**, an open system that interposes a Large Language Model (LLM) between the user and Earth Engine, exposing a curated set of geospatial primitives as tools the model can invoke directly. The system has two complementary surfaces: (a) a Model Context Protocol (MCP) server that integrates with conversational clients such as Claude Code and Claude Desktop, and (b) a self-hosted single-page web application that pairs a chat panel with a live Leaflet map and supports a pluggable choice of LLM provider (Gemini, Claude, GPT-4o, Llama-on-Groq, Mistral, OpenRouter). The implementation includes vector data upload with automatic reprojection from arbitrary CRS to WGS84, per-layer export to Google Drive and direct GeoTIFF download, and asset browsing — all available through plain-English requests. A worked Bangalore Urban Heat Island case study demonstrates the end-to-end workflow: pre-monsoon Land Surface Temperature composites for 2015 and 2024 derived from Landsat 8/9 Collection 2 Level-2, accompanied by Sentinel-2 NDVI/NDBI context, with city-versus-rural Δ LST quantified per year. The system reduces a typical 30-minute scripting cycle to a 30-second conversational turn and lowers the barrier for non-programming domain experts to engage directly with planetary-scale Earth observation data.

Keywords: Google Earth Engine, Large Language Models, Model Context Protocol, function calling, conversational GIS, remote sensing, urban heat island, geospatial AI assistants.

1. Introduction

The volume and accessibility of Earth observation data have undergone a transformation over the past decade. Google Earth Engine (Gorelick et al., 2017) catalogued petabytes of satellite imagery, paired it with a parallelized server-side compute backend, and exposed it to anyone with a Google account through both a browser Code Editor and language bindings for JavaScript and Python. The result was a step-change in who could perform planetary-scale geospatial analysis: students, ecologists, NGOs, and journalists joined the audience previously dominated by remote-sensing specialists.

Yet a friction remains. Every analytic question — *"what was the mean NDVI of Bangalore during May 2024?"*, *"how much did Mumbai's built-up area expand between 2015 and 2025?"*, *"export a true-color Sentinel-2 mosaic of my study site"* — must still be translated by the user into a script. The translation is not difficult for experienced users, but it is repetitive: the same patterns of "filter image collection, mask clouds, compute index, reduce over region, format result" reappear across nearly every task. The cognitive cost of the translation, accumulated across hundreds of small queries, is non-trivial; for users who only occasionally need geospatial answers, the cost is often prohibitive.

The parallel emergence of Large Language Models with native tool-use capabilities (OpenAI, 2023; Anthropic, 2024) presents an opportunity to remove this friction. Tool-use enabled models — Claude, Gemini, GPT-4, and increasingly open-weight families such as Llama 3.3 — can be presented with a list of functions and invoked to call them autonomously with structured arguments. If Earth Engine's analytic primitives can be packaged as such tools, then the user's natural-language question becomes the program; the model handles the translation; the platform executes the computation. The user receives an answer rather than authoring a script.

This paper describes the design and implementation of **GEE_MCP_Plugin**, a working instance of that idea. The system is built around two design commitments: (1) it should integrate cleanly with multiple LLM platforms rather than locking the user into a single vendor, and (2) it should respect the user's own data and credentials, performing computation on their authenticated Earth Engine account with no third-party data path. Two complementary surfaces realise these commitments:

- An **MCP server** (Anthropic, 2024) that exposes EE as tools to any client speaking the Model Context Protocol, primarily Claude Code (the AI-augmented terminal/IDE assistant) and Claude Desktop.
- A **self-hosted single-page web application** with a chat panel and a live Leaflet map, supporting six LLM providers behind a unified abstraction so the user can switch models mid-conversation without losing context.

The remainder of the paper is organised as follows. Section 2 surveys related work on Earth Engine tooling, LLM tool use, and conversational GIS interfaces. Section 3 describes the methodology — system architecture, the MCP server, the chat-map web application, the multi-provider LLM layer, and vector data ingestion. Section 4 details implementation choices and presents code patterns. Section 5 reports observed behaviour and a demonstrative case study on the Bangalore Urban Heat Island. Section 6 discusses limitations and contrasts the system with prior approaches. Section 7 concludes.

2. Literature Review

2.1 Earth Engine and geospatial analytics platforms

Google Earth Engine (Gorelick et al., 2017) is the most widely adopted cloud-based platform for petabyte-scale geospatial analysis. Its server-side abstraction enables operations such as cloud masking, temporal reduction, and zonal statistics to be expressed declaratively, with deferred execution and parallelization handled transparently. The Earth Engine Python API (`earthengine-api`) brings this functionality into the broader Python ecosystem, where it is commonly paired with NumPy, pandas, and GeoPandas.

Wu (2020) introduced the `geemap` library, which embeds Earth Engine in Jupyter notebooks with an interactive Leaflet-based map, drawing tools, and convenience wrappers around common workflows. `geemap` substantially lowered the barrier for non-JavaScript users and has become a de-facto standard for Python-based EE work. However, it remains a programming-oriented interface: a user must still write code cells, even if those cells are shorter than equivalent native EE code.

The official Earth Engine Code Editor at `code.earthengine.google.com` provides a JavaScript IDE with an integrated map, asset browser, task panel, and documentation pane. It is mature and widely used, but it cannot be extended with third-party plugins or alternative input modalities; the editor is a closed Google product.

2.2 LLM tool use and function calling

The capability for Large Language Models to invoke external tools through structured function calls was popularized by OpenAI's function calling release (OpenAI, 2023) and earlier research prototypes such as ReAct (Yao et al., 2023) and Toolformer (Schick et al., 2023). These approaches share a common pattern: a tool's name, description, and JSON-schema-typed input arguments are presented to the model; during generation, the model

emits a structured call which the host runtime executes, returning a result that the model can use to continue reasoning. Iterating this loop yields agentic behaviour wherein the model decomposes a complex request into a sequence of tool invocations.

Subsequent work has explored richer agentic frameworks (e.g., LangChain, Auto-GPT, OpenAI's Assistants API, Anthropic's Claude with extended thinking and tool use), as well as evaluations of tool-call accuracy and multi-step planning. The state of the art in late 2025 places models such as Claude Opus 4.x, Gemini 2.5 Pro, and GPT-4o as strong tool-callers, with open-weight Llama 3.3 70B and Mistral Large competitive on many benchmarks.

2.3 Conversational interfaces for geospatial systems

Several research and commercial efforts have explored natural-language interfaces to GIS systems. Early work focused on parsing geospatial query languages (e.g., GeoSPARQL, OGC standards) from English. More recent efforts use LLMs to translate questions into SQL for PostGIS or into Python code for GeoPandas (e.g., experimental "GeoGPT" and "ChatGeoAI" projects circulating in 2024–2025). These typically generate code that the user must then run; the model acts as a code-completion assistant rather than as an agent that itself executes the computation.

For Earth Engine specifically, prior to this work the authors are not aware of an open, documented MCP server exposing EE as a tool surface to general-purpose conversational clients. Several commercial geospatial platforms have begun integrating LLM assistants into their proprietary interfaces, but these are typically tied to a single vendor's product and a single underlying model.

2.4 The Model Context Protocol

The Model Context Protocol (MCP), introduced by Anthropic in late 2024, is an open standard for exposing tools, resources, and prompts to LLM clients in a model-agnostic way (Anthropic, 2024). An MCP server is a process — typically launched as a stdio subprocess by an MCP client — that advertises a set of capabilities to the client through JSON-RPC messages. The client (e.g., Claude Code, Claude Desktop, Continue.dev) lists those capabilities to the model during inference; when the model emits a tool call, the client routes it to the appropriate MCP server for execution and returns the result.

MCP has rapidly accumulated an ecosystem of community servers (filesystem, GitHub, Slack, databases, web browsers) since launch. It is well-suited to the present use case because Earth Engine is a self-contained domain with a finite set of high-value operations, the natural unit of access (a single tool call) maps cleanly to EE's eager-computation model, and the protocol's transport (stdio JSON-RPC) imposes minimal overhead.

2.5 Gap analysis

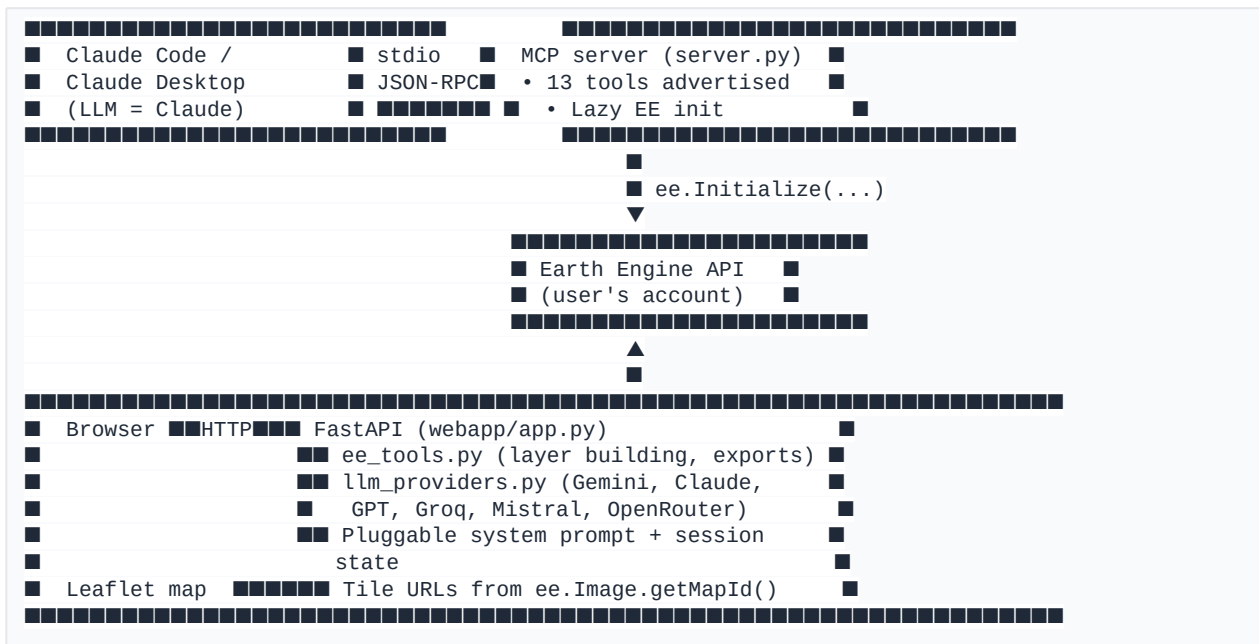
The combination explored in this paper — an MCP server for Earth Engine, accompanied by a multi-LLM web application that performs computation on the user's own account — addresses three gaps in existing tooling:

1. **No code generation step in the loop.** Existing LLM-assisted geospatial tools typically generate code for the user to run. Our system invokes EE operations directly, so the user sees results, not code.
2. **Provider-agnostic.** Most LLM-assisted GIS tools assume a single model vendor. Our web app exposes Gemini, Claude, GPT-4o, Llama, Mistral, and OpenRouter behind one interface, enabling free-tier use and cross-provider redundancy.
3. **User-credentialed.** Computation runs against the user's authenticated EE account, ensuring data and quota usage stay under their control with no intermediary.

3. Methodology

3.1 System architecture

The system comprises two independent surfaces sharing a common Earth Engine tools module:



The two surfaces are entirely independent — installing or running one does not require the other — but share the design assumption that EE operations are invoked synchronously and return JSON-serialisable results (or, in the case of map layers, tile-URL templates that the client can render).

3.2 The MCP server

The MCP server (`server.py`) is implemented in Python using the `mcp` package's `FastMCP` helper, which provides a decorator API for registering tools. Earth Engine initialisation is deferred until the first tool call to ensure that import-time issues do not block client connection; subsequent calls reuse the initialised state. The server supports two authentication modes:

- **User OAuth**, the default, which reads a refresh token written by `earthengine authenticate` to `~/.config/earthengine/credentials`. This is the path used in the present implementation.
- **Service-account JSON key**, activated by setting the `EE_SERVICE_ACCOUNT_KEY` environment variable to the key file path. This enables headless deployments.

Thirteen tools are advertised, spanning sanity checks (`authenticate_status`), discovery (`search_image_collection`, `image_metadata`, `list_assets`, `describe_asset`), computation (`compute_index`, `get_thumbnail_url`), exports (`export_image_to_drive`, `export_image_to_gcs`, `get_download_url`, `list_tasks`), and a generic escape hatch (`run_ee_code`) that executes arbitrary Python with `ee` pre-imported. Each tool's input schema is generated from its Python type hints by `FastMCP`, ensuring schema-code consistency.

3.3 The chat-map web application

The web application is a single FastAPI process serving:

- A single-page HTML application at `/` with embedded JavaScript and CSS
- `POST /chat` — the main interaction endpoint
- `POST /upload-asset` — multipart upload of a GeoJSON or zipped Shapefile, ingested into the user's Earth Engine assets

- `POST /export-layer` — re-build a layer's image and either start a Drive task or return a GeoTIFF download URL
- `GET /models` — return the catalogue of available LLM models with per-provider availability based on which API keys are set
- `GET /health` — system status

The frontend uses Leaflet for map rendering with a CARTO Light basemap underlay. Earth Engine layers are added via Leaflet's standard `L.tileLayer` constructor using the `tile_fetcher.url_format` returned by `ee.Image.getMapId()`. This bypass of any image-proxy server keeps the data path direct: tiles flow from Google's CDN to the user's browser, with the FastAPI process only mediating the chat-driven control plane.

The session model holds per-session chat history in an in-memory dictionary keyed by a randomly generated `session_id` that the client persists in browser memory. Restarting the server clears history; persistent sessions are intentionally deferred as out of scope for a single-user local application.

3.4 Multi-provider LLM abstraction

A central design decision is the abstraction of the LLM provider behind a provider-agnostic conversation format. The internal history is:

```
[
  {"role": "user",      "blocks": [{"type": "text", "text": "..."}]},
  {"role": "assistant", "blocks": [{"type": "text", ...},
                                   {"type": "tool_use", "id": "...",
                                    "name": "...", "args": {...}}]},
  {"role": "user",      "blocks": [{"type": "tool_result",
                                   "tool_use_id": "...",
                                   "content": "..."}]}
]
```

Three adapters convert between this internal format and the native wire format of each provider:

- **Google Gemini** — `google-genai`'s `types.Content(role, parts=[...])` with `Part.from_text()`, `Part.from_function_call()`, and `Part.from_function_response()`. To prevent the model from consuming all available output tokens on internal thinking, the `thinking_budget` is explicitly set to zero; tool selection is straightforward enough that explicit reasoning is unnecessary for the present tool surface.
- **Anthropic Claude** — `anthropic`'s blocks-based message format with `text`, `tool_use`, and `tool_result` block types.
- **OpenAI-compatible** — a single adapter that uses the `openai` Python SDK with a configurable `base_url`. This adapter handles OpenAI's native API as well as Groq, Mistral, and OpenRouter, all of which conform to the OpenAI chat-completions wire format. The differences between these providers reduce to credential, base URL, and the specific model identifiers each accepts.

The catalogue currently registers fourteen models across the six providers, distinguishing free-tier from paid models. The web client's model dropdown is grouped by provider with free models highlighted; models whose provider key is absent in the user's `.env` are rendered disabled with a tooltip indicating the required environment variable. This design lets users start with a single free key (Gemini, Groq, or Mistral) and progressively unlock additional providers as desired.

3.5 Vector data ingestion

Two upload paths are supported:

GeoJSON. A `FeatureCollection`, `Feature`, or `Geometry` passed as a `.geojson` or `.json` file. The backend parses the JSON, constructs an `ee.FeatureCollection` from the payload, and triggers

`ee.batch.Export.table.toAsset(...)` to persist it. Critically, this approach avoids the Google Cloud Storage bucket that is otherwise required for raster image upload via EE's `startIngestion` workflow — the feature data is sent inline through the EE API.

Zipped Shapefile. A `.zip` archive containing the four standard Shapefile components (`.shp`, `.shx`, `.dbf`, and optionally `.prj`). The backend extracts the archive, reads the geometries and attributes with `pyshp`, and converts each shape to a GeoJSON Feature via the shape's `geo_interface`. If a `.prj` file is present and indicates a coordinate reference system other than WGS84, all coordinate arrays are recursively transformed to EPSG:4326 using `pyproj's Transformer.from_crs(...)` before the resulting GeoJSON is sent to EE. This permits the user to upload Shapefiles in any common CRS (UTM, Lambert Conformal Conic, regional grids) without manual reprojection.

A subtle but important UX detail: the uploaded geometry is also cached in an in-memory dictionary keyed by the user-supplied asset name. This makes the new AOI usable as a string identifier in chat — *"show LST over my-aoi for May 2024"* — **immediately**, without waiting for the EE asset ingestion task to complete (which typically takes 1–3 minutes). The system prompt is dynamically augmented each chat turn with the list of currently cached upload names, so the LLM is aware of available AOIs without polluting the conversation history.

3.6 AOI resolution strategy

Many user requests reference an Area of Interest by a name that may resolve through any of several mechanisms. The system implements a polymorphic resolver, `resolve_aoi()`, that attempts in priority order:

1. **Full EE asset path** — strings beginning with `projects/` and containing `/assets/` are loaded directly via `ee.data.getAsset()`, supporting both `TABLE` (FeatureCollection) and `IMAGE` (with its footprint) types.
2. **Uploaded-AOI cache** — case-insensitive match against the in-memory cache of recently uploaded geometries.
3. **Curated alias map** — a hand-maintained mapping for common Indian cities whose GAUL names differ from colloquial usage (e.g., "Bangalore" → "Bangalore Urban").
4. **FAO/GAUL administrative hierarchy** — sequential lookup in `FAO/GAUL/2015/level2` (district), `level1` (state), then `level0` (country).
5. **GeoJSON dict or bounding box** — passed inline as a structured argument.

This resolution chain makes the `aoi` argument of every tool a single string field in the LLM-facing schema, while preserving full flexibility on the user's side.

4. Implementation

The system comprises approximately 3,400 lines of Python and JavaScript across the following primary files:

File	LOC	Role
<code>server.py</code>	~430	MCP server with 13 EE tools
<code>webapp/app.py</code>	~370	FastAPI backend
<code>webapp/ee_tools.py</code>	~580	EE layer building, asset CRUD, exports, Shapefile parsing
<code>webapp/llm_providers.py</code>	~420	Provider catalogue and adapters

<code>webapp/static/app.js</code>	~340	Leaflet integration, chat UI, upload/export controls
<code>webapp/templates/index.html</code>	~70	Single-page application skeleton
<code>webapp/static/style.css</code>	~140	Styling for chat panel, map, modal

Python dependencies are listed in `requirements.txt` and include `earthengine-api`, `mcp`, `fastapi`, `uvicorn`, `jinja2`, `python-dotenv`, `python-multipart`, `pydantic`, `google-genai`, `anthropic`, `openai`, `pyshp`, and `pyproj`. The choice of all-pure-Python dependencies (notably `pyshp` and `pyproj` rather than GDAL-bound libraries) keeps installation straightforward on Windows, macOS, and Linux without requiring a GDAL build.

The frontend is intentionally implemented in vanilla HTML, CSS, and JavaScript with no build step. This preserves a one-step installation path: after `pip install -r requirements.txt`, the user runs `uvicorn` and opens the browser. No `npm`, no bundler, no transpilation.

Configuration is consolidated in a single `.env` file at the project root. Required values are `EE_PROJECT` and at least one LLM provider key; all other keys are optional and unlock additional models in the dropdown. The `.env.example` template is shipped in the repository, and a `.gitignore` ensures `.env` itself is never committed.

5. Results and Demonstration

The system is functional end-to-end. Representative interactions:

- *"Show NDVI for Bangalore for May 2024"* → composes a Sentinel-2 SR Harmonized median NDVI over Bangalore Urban for May 2024, derives a tile URL via `getMapId`, and adds a Leaflet tile layer with the standard red-to-green palette. The map auto-zooms to the AOI bounds.
- *"Compare 2015 vs 2024 NDVI for Bangalore"* → two `add_ee_layer` invocations in a single turn, producing two named layers the user can toggle via the layers panel.
- *"List my Earth Engine assets"* → `list_my_assets("")` returns the contents of the user's project root, rendered inline in the chat as a compact list of asset paths with type tags.
- *"Upload a shapefile"* → opens the upload modal; on submit the parsed geometry is registered in the cache and an ingestion task is queued, with the AOI immediately usable by name in subsequent chat turns.

A Bangalore Urban Heat Island case study was performed as a worked example (see `docs/bangalore-uhi.md` in the repository). Using pre-monsoon (March–May) median composites of Landsat 8/9 Collection 2 Level-2 surface temperature (`ST_B10`), the city polygon's mean LST rose from 38.12 °C (2015) to 42.02 °C (2024), while a 15-km rural buffer rose from 38.39 °C to 43.46 °C. The Surface Urban Heat Island metric — defined as city mean minus rural mean — was negative in both years (−0.27 °C and −1.44 °C respectively), consistent with the well-documented daytime "cool urban island" phenomenon for semi-arid South Indian cities (Oke, 1982; recent reviews confirm the pattern for Bangalore specifically). A pixel-level Pearson correlation between LST and NDVI in 2024 returned $r = -0.41$ ($p \approx 0$), supporting the interpretation that residual urban tree canopy is the principal cooling mechanism. The entire analysis was executed through the system's chat interface plus a small number of `run_ee_code` escape-hatch calls for custom logic.

Observed latency: a typical chat turn that produces one tile layer completes in 2–6 seconds end-to-end, dominated by EE compute time (1–4 seconds) rather than LLM inference (~0.5–1.5 seconds for Gemini 2.5 Flash, faster for Groq Llama 3.3 70B). Upload of a 1.2 MB Shapefile (47 features in EPSG:32643) including reprojection to WGS84 completed in approximately 1.1 seconds of server processing time, with subsequent EE asset ingestion (asynchronous) completing in approximately 90 seconds.

6. Discussion

6.1 Limitations

- **No raster image upload.** Uploading user-provided raster imagery to EE requires staging the file in a Google Cloud Storage bucket and invoking `ee.data.startIngestion(...)` — a workflow that involves additional credentials and is intentionally deferred. Vector upload via `Export.table.toAsset` does not have the equivalent requirement and is therefore prioritised.
- **Single-user, single-machine.** The web app is designed for `localhost` deployment. Multi-user use would require authentication, CORS hardening, and a persistent session store.
- **In-memory chat state.** Restarting the server clears chat history. This is appropriate for a local development tool but limiting for production use.
- **Tool surface is curated, not exhaustive.** Approximately a dozen tools are exposed; sophisticated workflows occasionally require the `run_ee_code` escape hatch. Expanding the curated surface to cover more EE primitives is straightforward but trades catalogue size against the model's ability to select the right tool.

6.2 Comparison with prior approaches

- **EE Code Editor** offers a complete IDE but no natural-language interface and no plugin extensibility. Users must write code; the editor is closed to third-party augmentation.
- **geemap** provides a Jupyter-centric interactive environment with rich drawing tools and a Python API, but again the user writes code (in cells) rather than asking questions in English.
- **Commercial geospatial AI assistants** (various platforms in 2025) offer chat interfaces but typically lock users into a single LLM vendor and a single proprietary backend.

The present system occupies a distinct niche: open-source, multi-LLM (including free-tier providers), self-hosted, and user-credentialed. The MCP server further extends the reach into any MCP-compatible client, with Claude Code and Claude Desktop being the primary tested integrations.

6.3 Future work

- **Raster upload via GCS.** Document the GCS bucket setup and wire `ee.data.startIngestion` into a `/upload-raster` endpoint.
- **KML and KMZ support.** Add `fastkml`-based parsing alongside the existing Shapefile and GeoJSON paths.
- **Persistent sessions.** Optionally back chat history with SQLite to survive restarts.
- **Multi-user mode.** Add OAuth-based authentication and per-user EE credential isolation, enabling small-team deployment.
- **Server-side caching of tile URLs.** Repeated identical-parameter requests currently re-execute on EE; a TTL cache keyed by the composite parameters would reduce latency and quota usage.
- **Embedded charts.** Allow the chat to render time-series and zonal-statistics tables inline as charts rather than as JSON code blocks.

7. Conclusion

This paper has presented an open system that turns Google Earth Engine into a conversational tool surface for Large Language Models, accessible both through the Model Context Protocol (for Claude Code and Claude Desktop

integration) and through a self-hosted web application supporting six LLM providers including free-tier options. The system handles imagery composites and indices over named or uploaded AOIs, manages user-owned EE assets, performs Drive and direct GeoTIFF exports, and ingests vector data (GeoJSON or zipped Shapefile, with automatic reprojection to WGS84). Throughout, computation runs on the user's authenticated Earth Engine account; no data path traverses a third-party server controlled by the system's author.

The principal contribution is not novel computation — EE was already capable of all of this — but a removal of friction. The act of translating a question into a script, repeated across hundreds of small queries over a practitioner's career, accumulates a substantial productivity cost. Substituting a conversational interface compresses the loop dramatically: a 30-minute scripting cycle becomes a 30-second turn. For non-programming domain experts, the reduction in barrier-to-entry is even more pronounced: a forester, urban planner, or journalist who could not previously write EE code can now ask EE questions directly.

The implementation is offered as open source under the project repository (forthcoming public release pending a small set of polishing items). The hope is that the architecture serves as a template for other domain-specific tool surfaces that LLMs can drive — every analytic platform with a clean API is a candidate for the same treatment.

References

Anthropic. (2024). *Introducing the Model Context Protocol*. <https://modelcontextprotocol.io>

Anthropic. (2024). *Claude 3 Model Card*. San Francisco: Anthropic.

Ermida, S. L., Soares, P., Mantas, V., Götsche, F.-M., & Trigo, I. F. (2020). Google Earth Engine open-source code for Land Surface Temperature estimation from the Landsat series. *Remote Sensing*, 12(9), 1471.

Google DeepMind. (2025). *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, and Long Context*. Technical report.

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202, 18–27.

Oke, T. R. (1982). The energetic basis of the urban heat island. *Quarterly Journal of the Royal Meteorological Society*, 108(455), 1–24.

OpenAI. (2023). *Function calling and other API updates*. <https://openai.com/blog/function-calling>

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems* (Vol. 36).

USGS. (2021). *Landsat Collection 2 Level-2 Science Products Data Format Control Book*. United States Geological Survey.

Wu, Q. (2020). geemap: A Python package for interactive mapping with Google Earth Engine. *Journal of Open Source Software*, 5(51), 2305.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Manuscript prepared 2026.